The Ghost in the Heap

Defeating a Memory Thief

NICOLAS ARROYO





The Ghost in the Heap: Defeating a Memory Thief

Bloomberg

Engineering

CppCon 2026
September 16, 2026

Nicolas Arroyo
Software Architect, Order & Trade Capture
narroyo1@bloomberg.net

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

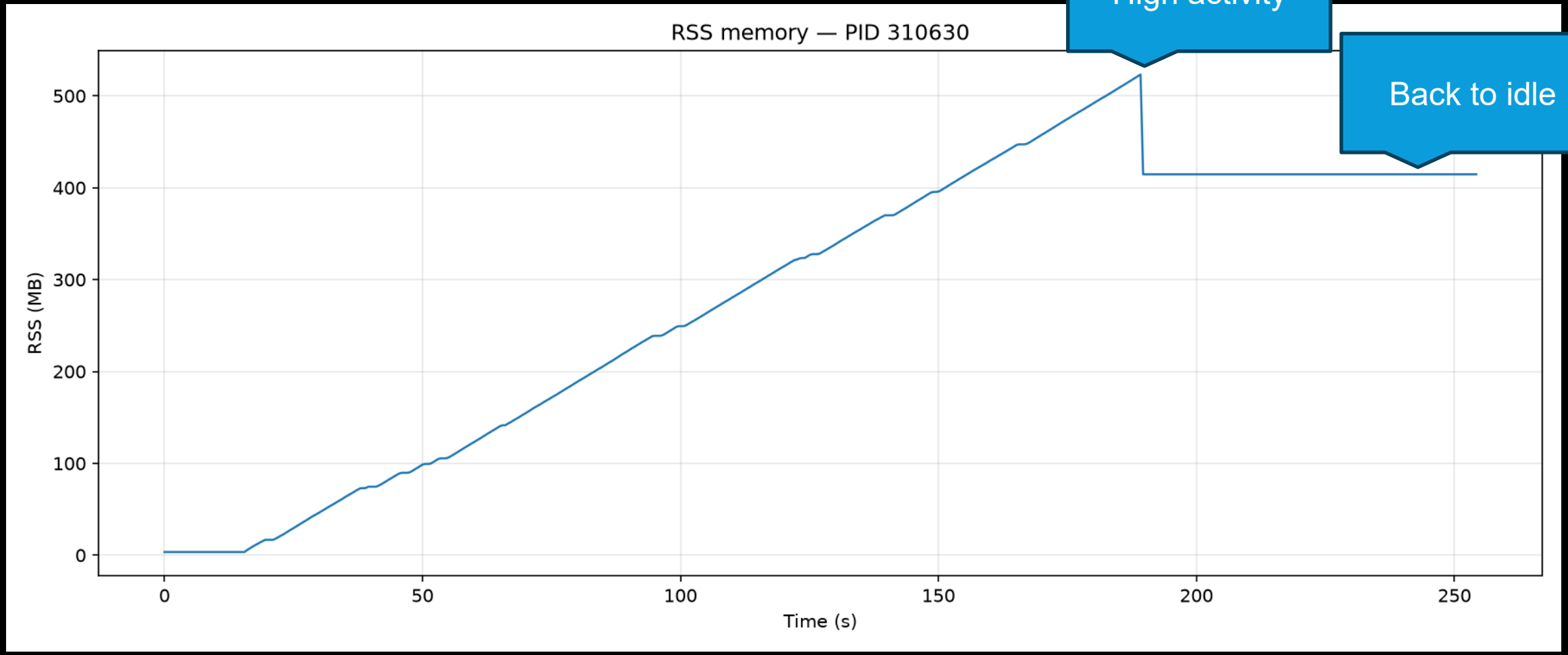
When Memory Won't Shrink

The Resident Set Size (RSS) Mystery

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

One-Way Memory Footprint Growth



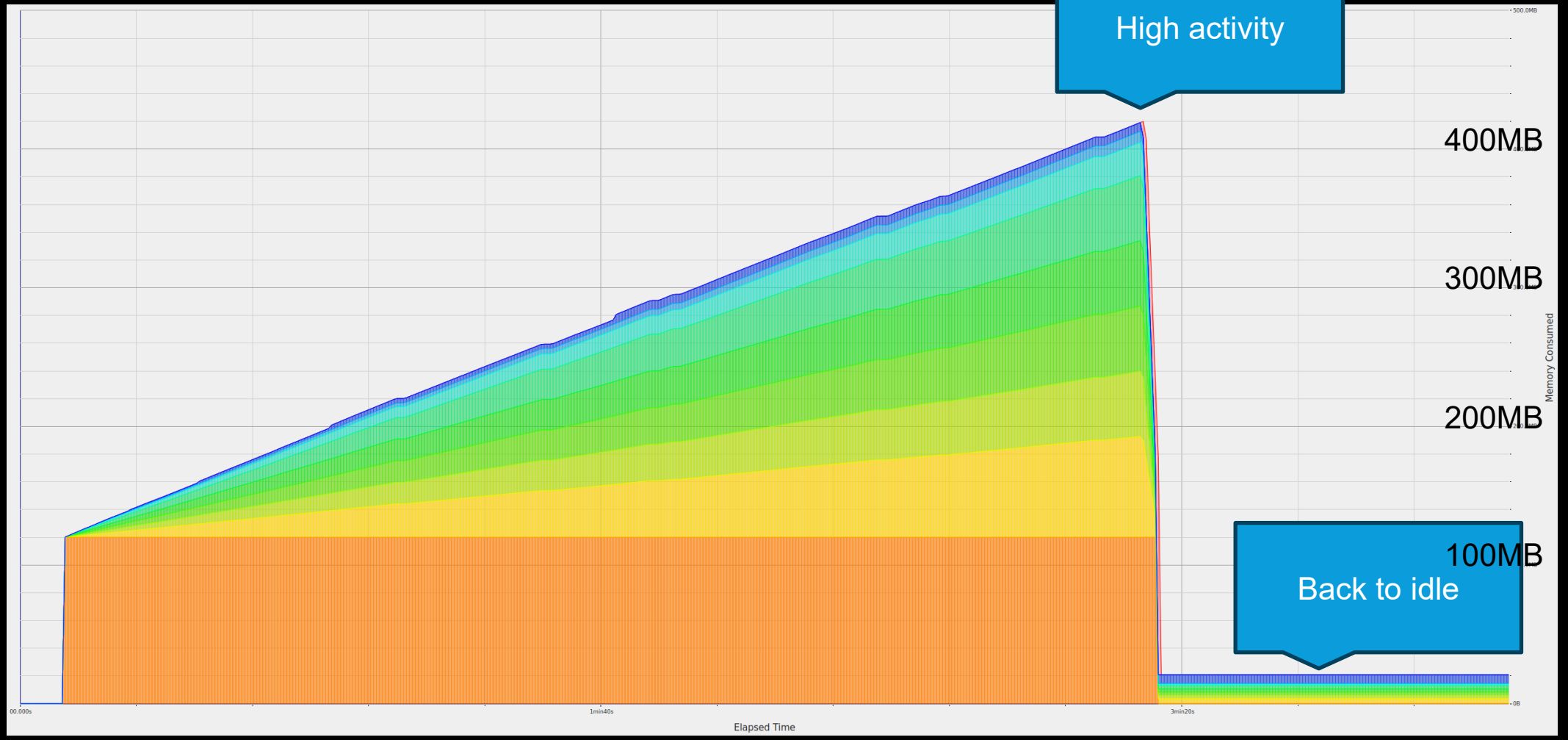
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

One-Way Memory Footprint Growth



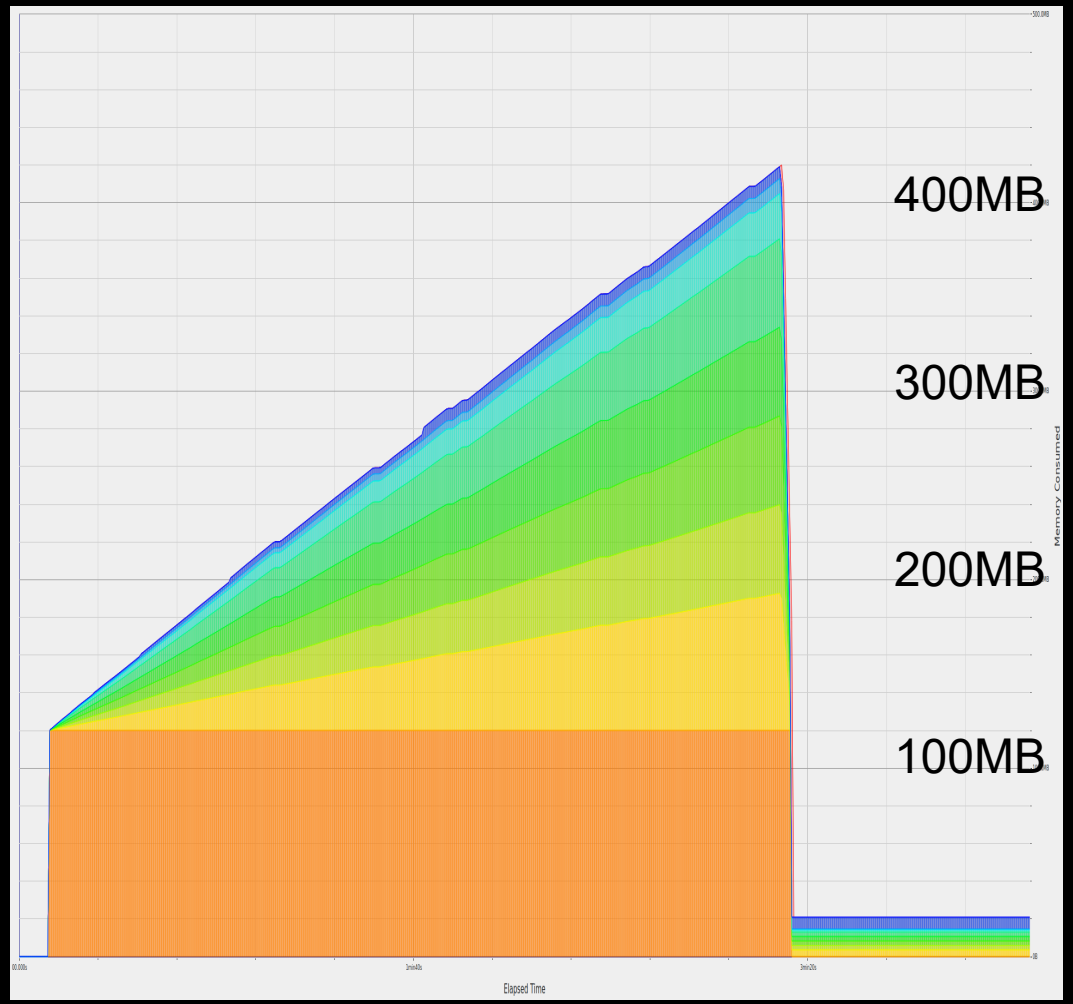
TechAtBloomberg.com

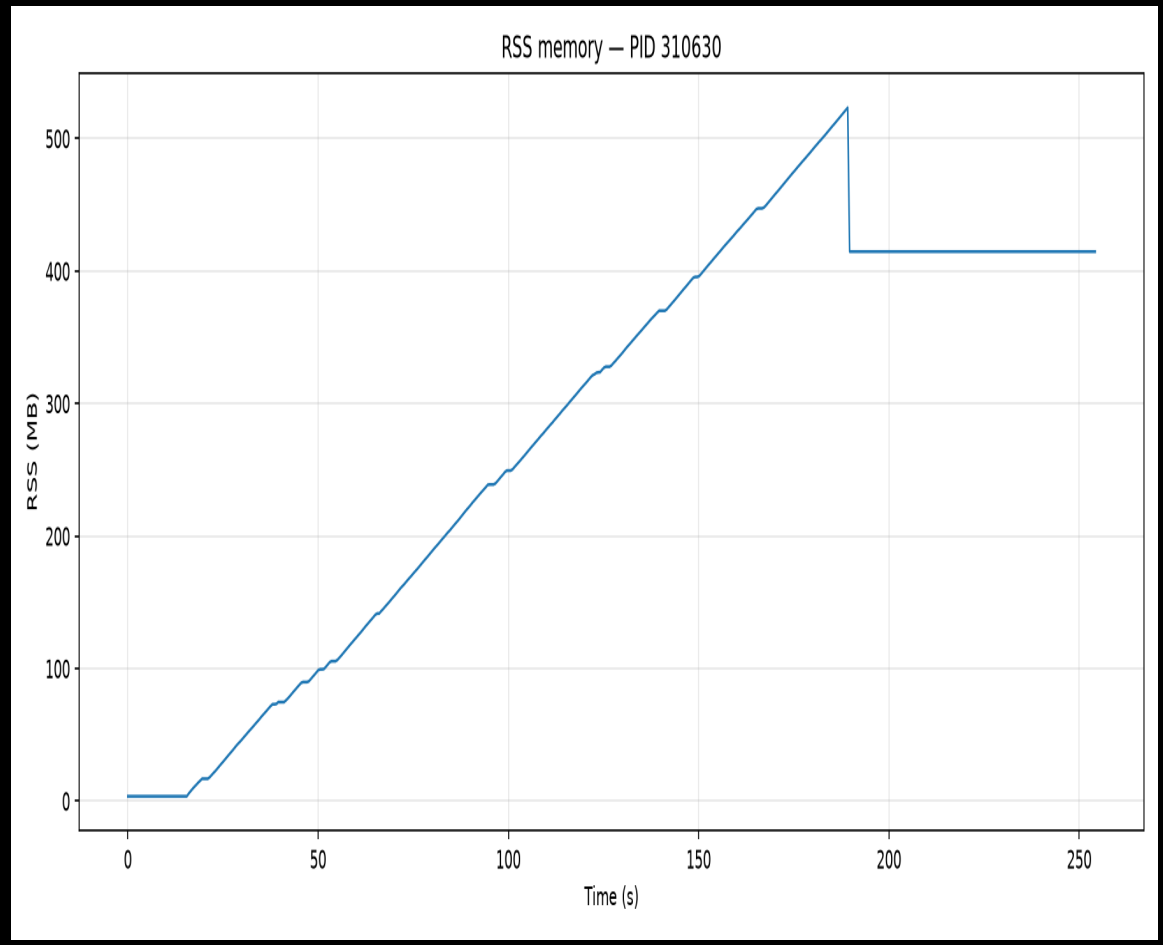
Bloomberg

Engineering

© 2026 Bloomberg Finance L.P. All rights reserved.

One-Way Memory Footprint Growth





TechAtBloomberg.com

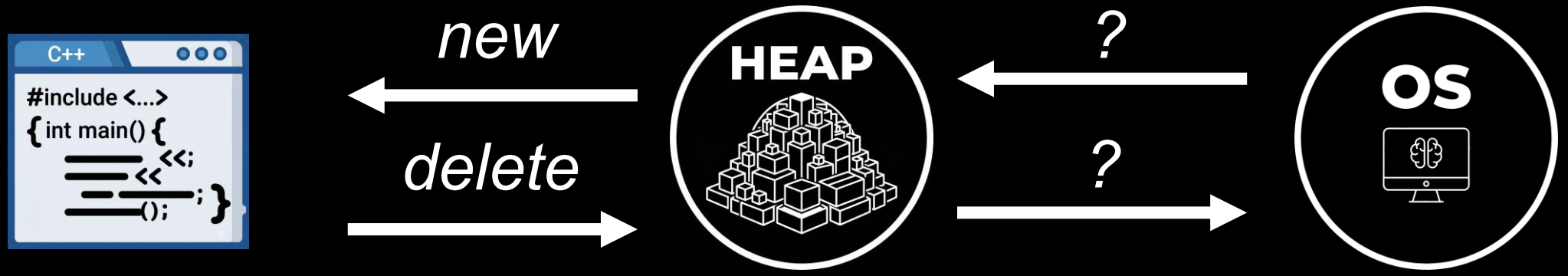
© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

The Goal: Reclaiming Physical Memory

- Excessive memory use can cause:
  - Swapping and Thrashing
  - Page Cache Starvation
  - OOM Kills
- Long-running services should be resource efficient



© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Time to Look Under *malloc()*'s Hood

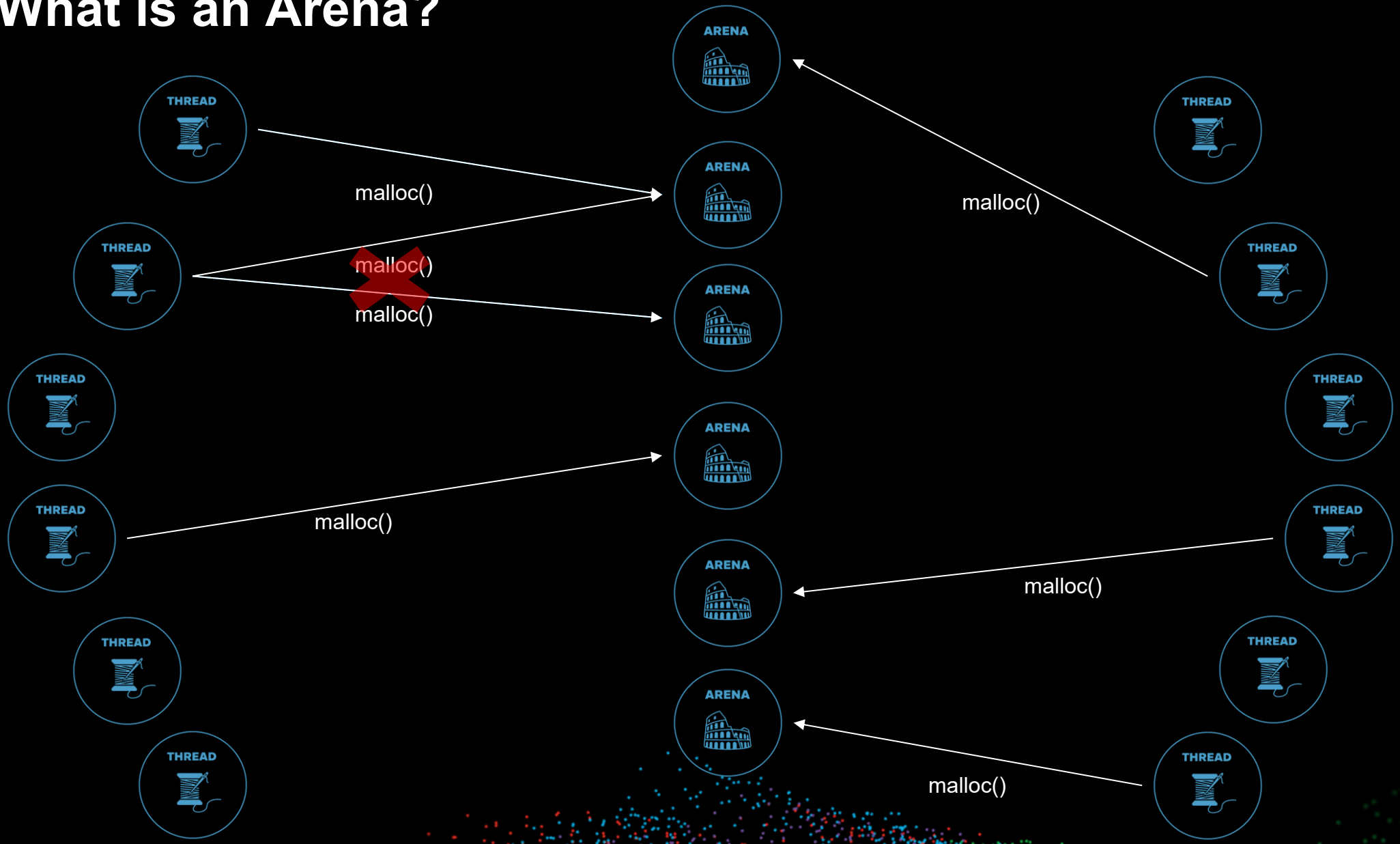
glibc / *ptmalloc2*



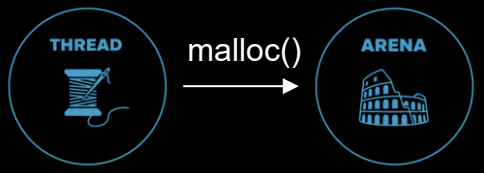
TechAtBloomberg.com

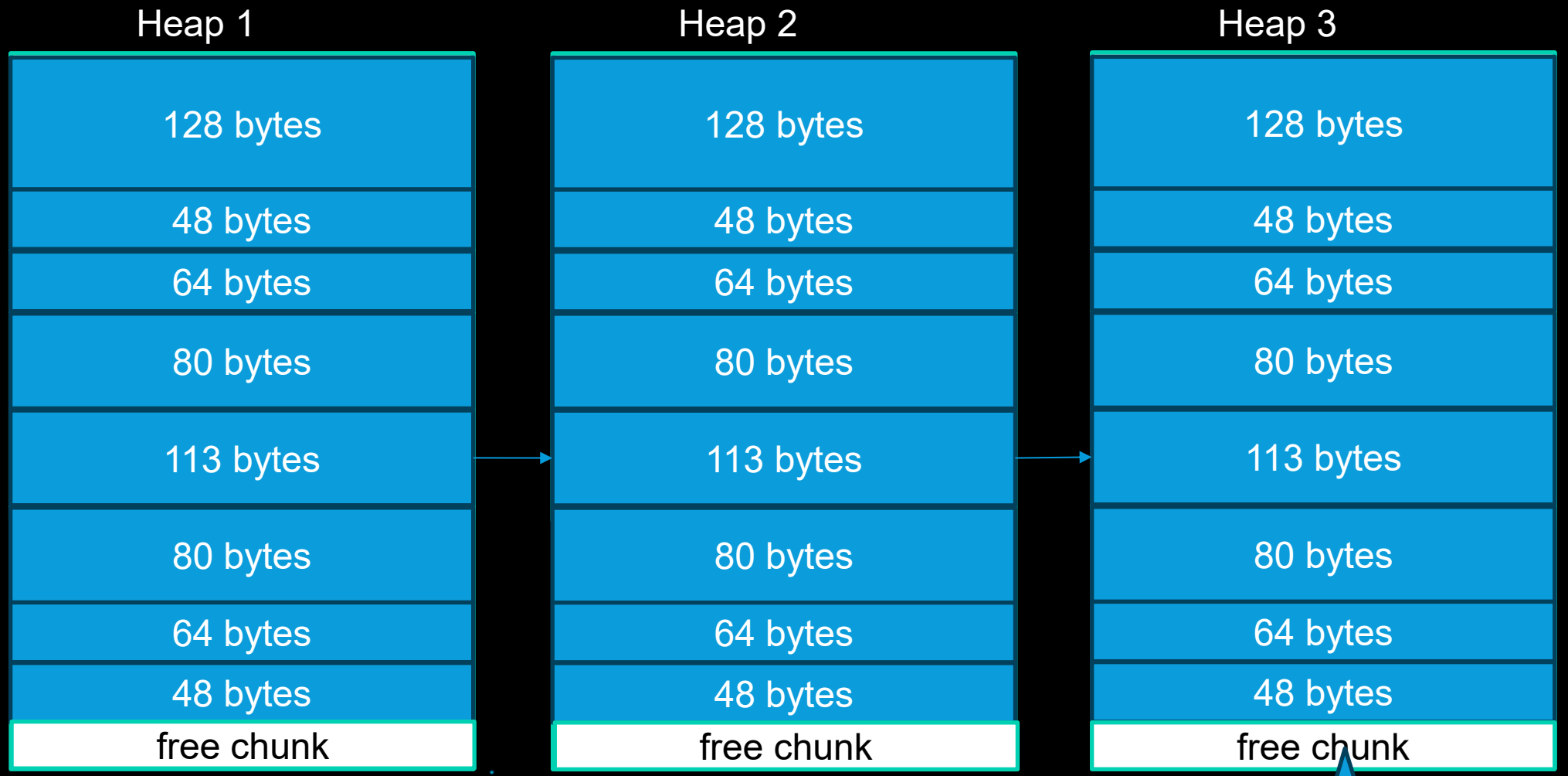
© 2026 Bloomberg Finance L.P. All rights reserved.

What is an Arena?

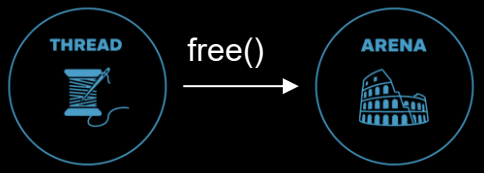


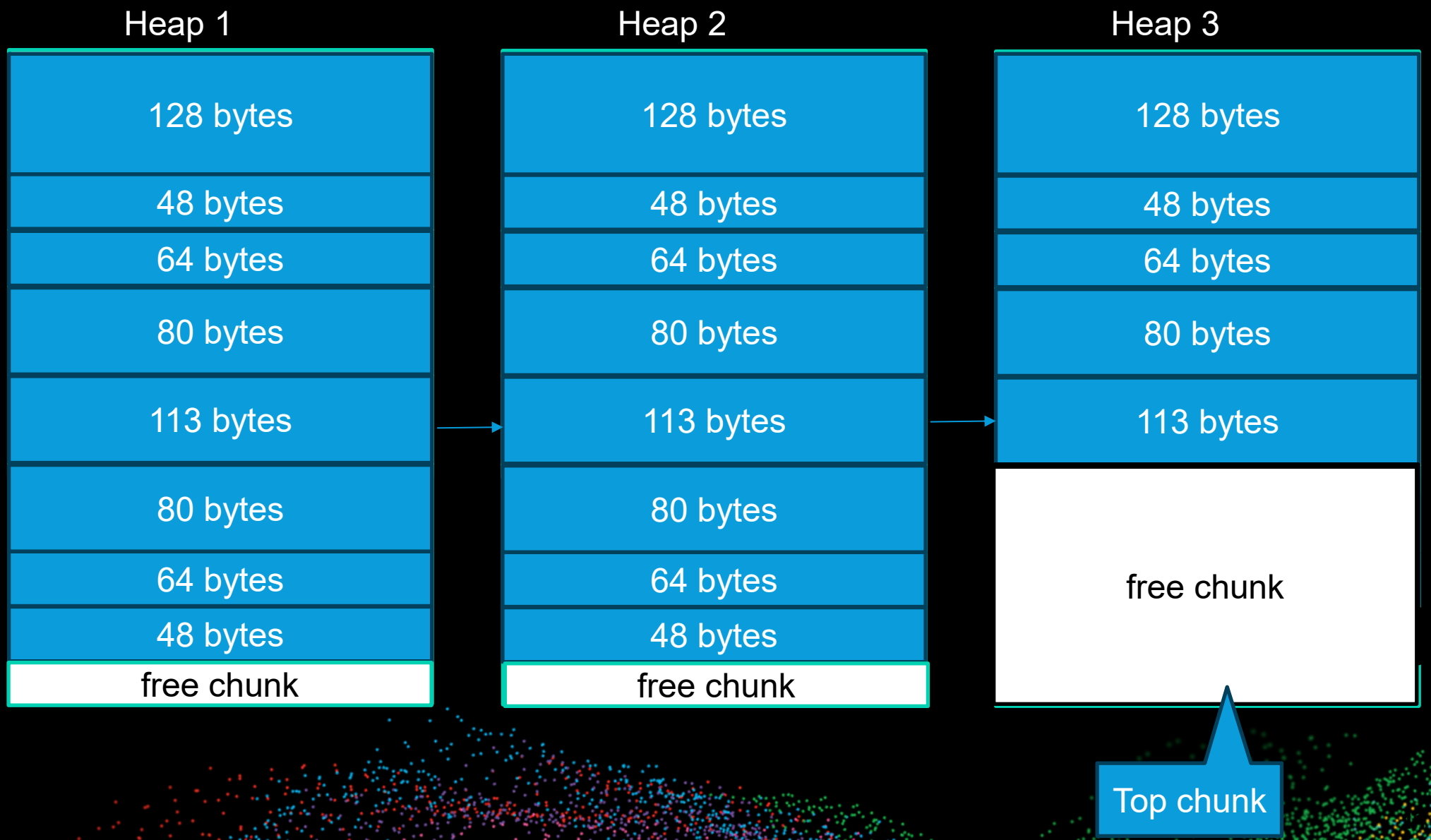
How do Arenas Grow?





How do Arenas Shrink?

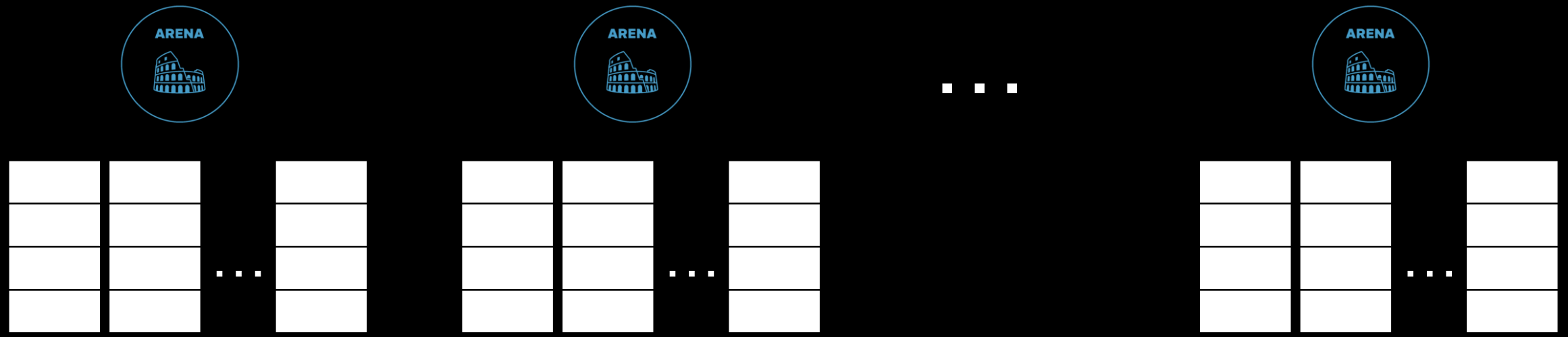




Why do Arenas Refuse to Shrink? The Mechanics of Pinning



How Multi-Threading Compounds the Problem



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

The Visibility Gap

Observing the Invisible

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Why do Heap Profilers Miss Heap Pinning?

- Always understand your profiling tools
- Most profilers see memory as a quantity
  - How much do the currently active allocations add up to?
  - How much memory was allocated by this code path?
- Allocation memberships and positions are also needed
  - Which allocations are pinning this heap?
  - What is the heap memory usage vs. footprint?

© 2026 Bloomberg Finance L.P. All rights reserved.

Let's Build a Positional Heap Profiler: Hooking *malloc() and* Friends

```cpp
auto sendRequest(const char* url, int timeout) -> int {
    Request* req = new Request{};
    std::strncpy(req->url, url, sizeof(req->url) - 1);
    req->timeout = timeout;
    int status = tcpSend(*req);
    delete req;
    return status;
}
```

new

delete

```cpp
auto malloc_hook(std::size_t size) -> v
    on_enter_allocation(size);
    auto result = std::malloc(size);
    on_leave_allocation(
    return result;
}

auto free_hook(void* ptr) -> void {
    on_leave_allocation(ptr);
    std::free(ptr);
}
```

Allocation size

Original function

Allocation address

Allocation address

Original function

- Intercept all malloc family functions
  - *LD_PRELOAD* or Hot-patch trampolines
- Read the requested size and returned address
- Collect the current callstack
  - Linux *backtrace, libunwind,* custom impl
- Forward calls to real library

```
#0  malloc_hook(size=264)                                  at cppcon_hook.cpp:44
#1  operator new(size=264)                                 at <libstdc++>
#2  sendRequest(url="https://api.example.com/data", timeout=5000) at cppcon_hook.cpp:17
#3  fetchMarketData(symbol="AAPL")                         at market_service.cpp:156
#4  updatePriceCache(symbols=...)                          at price_cache.cpp:89
#5  handleClientRequest(request=...)                       at request_handler.cpp:234
#6  workerThreadLoop()                                     at thread_pool.cpp:45
#7  std::thread::_State_impl::_M_run()                     at <libstdc++>
#8  start_thread()                                         at <pthread>
```

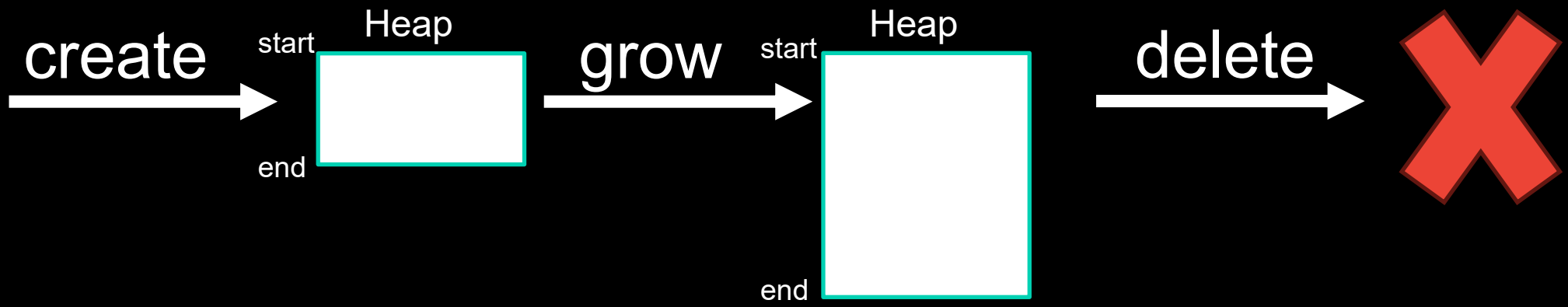
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

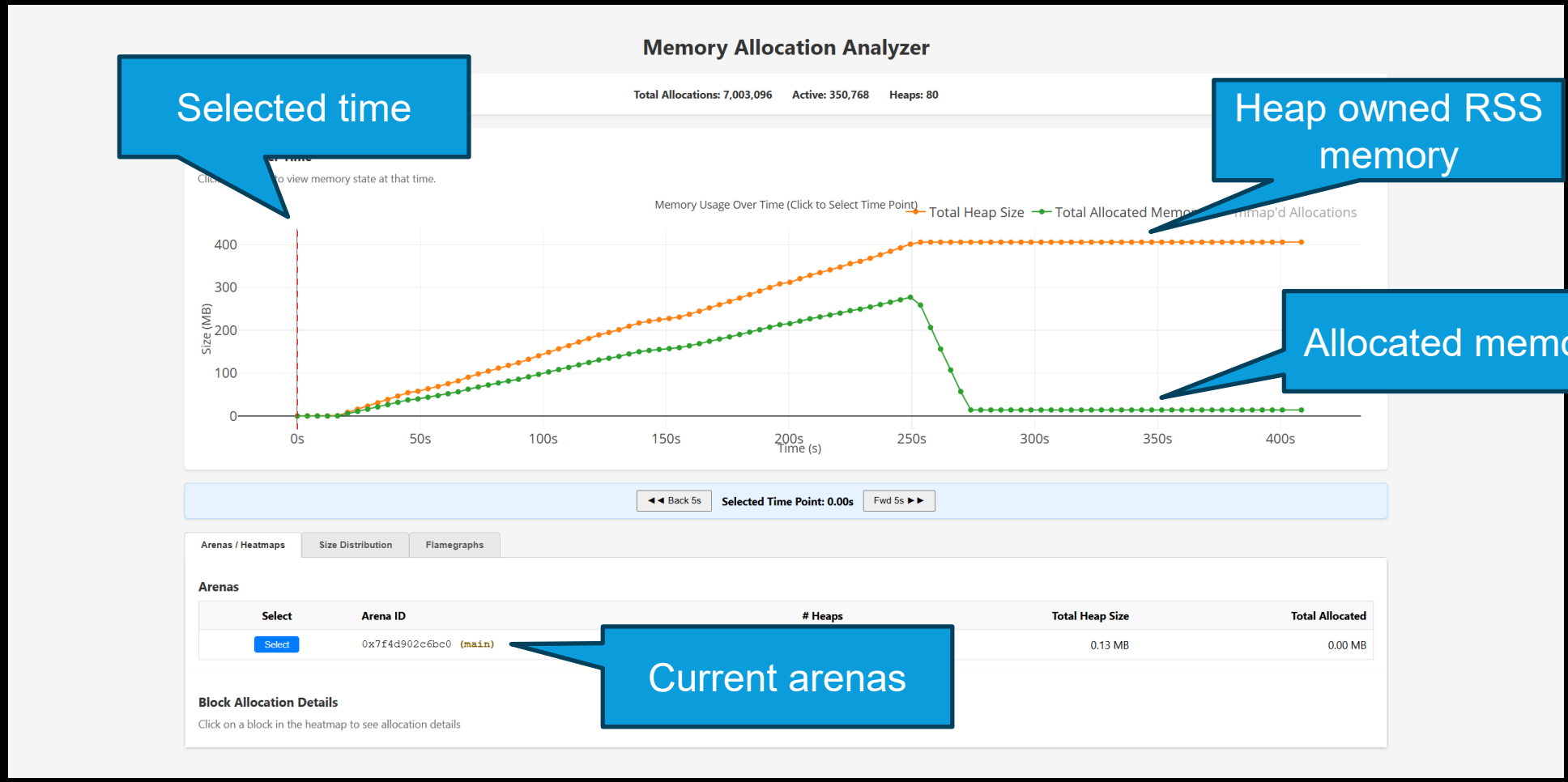
Let’s Build a Positional Heap Profiler: Observing Internal Structures



- Intercept arenas/heap creation/deletion/resizing
- Retain all allocation addresses
- Assign each allocation to a heap

© 2026 Bloomberg Finance L.P. All rights reserved.

Let’s Build a Positional Heap Profiler: Visualizing Pinning



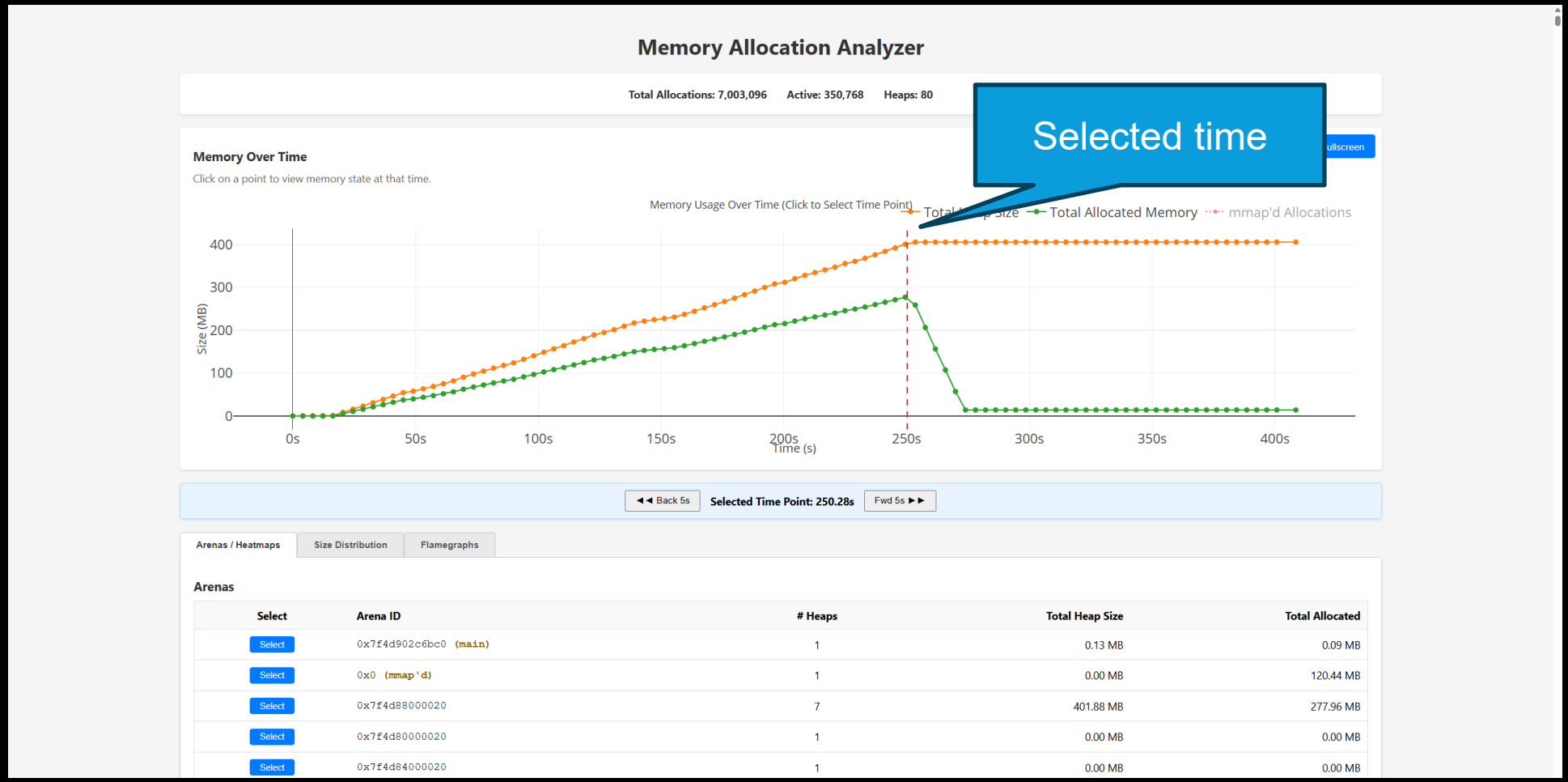
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



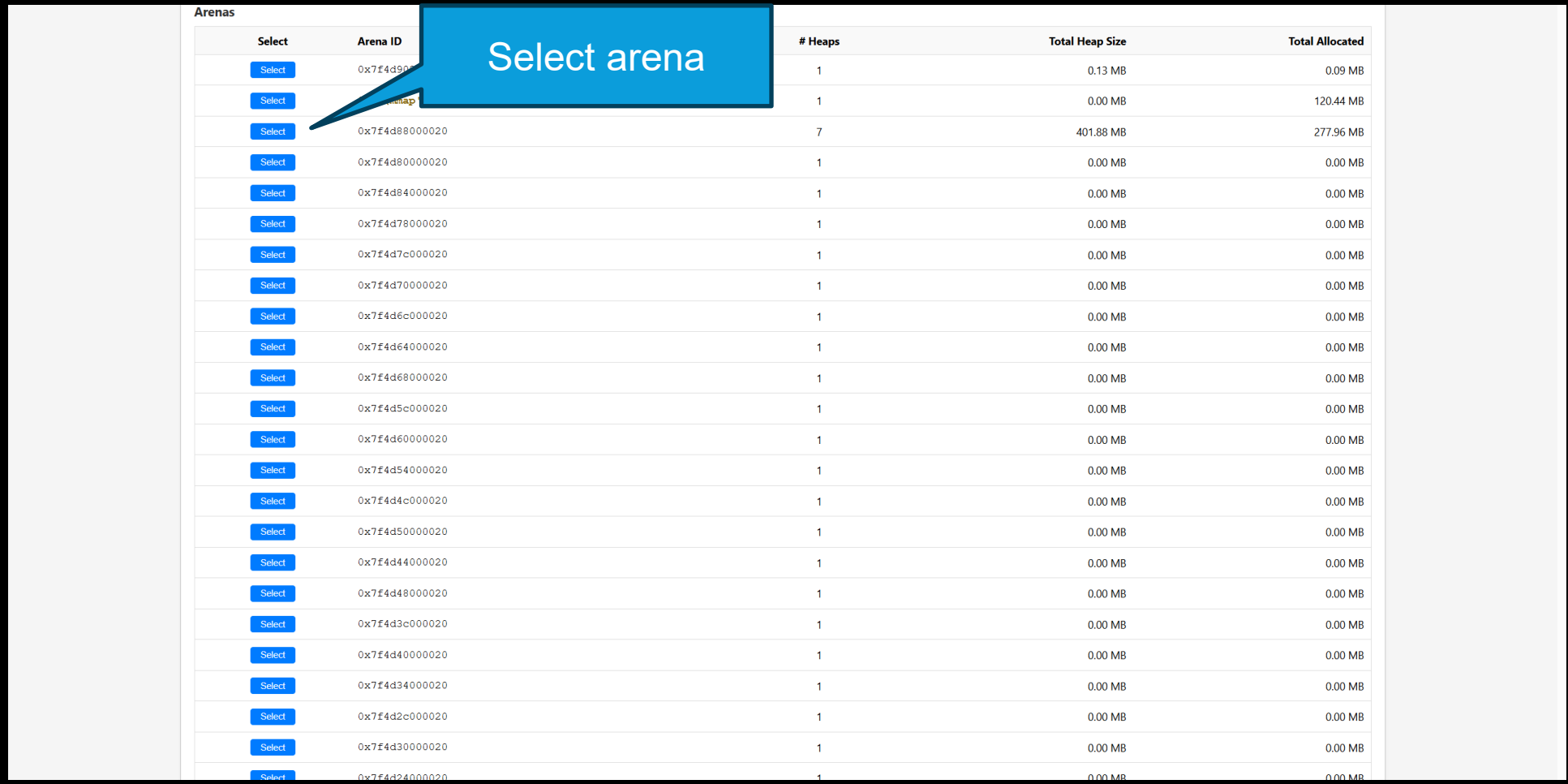
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



Arenas

Select	Arena ID	# Heaps	Total Heap Size	Total Allocated
Select	0x7f4d90	1	0.13 MB	0.09 MB
Select	[illegible]	1	0.00 MB	120.44 MB
Select	0x7f4d88000020	7	401.88 MB	277.96 MB
Select	0x7f4d80000020	1	0.00 MB	0.00 MB
Select	0x7f4d84000020	1	0.00 MB	0.00 MB
Select	0x7f4d78000020	1	0.00 MB	0.00 MB
Select	0x7f4d7c000020	1	0.00 MB	0.00 MB
Select	0x7f4d70000020	1	0.00 MB	0.00 MB
Select	0x7f4d6c000020	1	0.00 MB	0.00 MB
Select	0x7f4d64000020	1	0.00 MB	0.00 MB
Select	0x7f4d68000020	1	0.00 MB	0.00 MB
Select	0x7f4d5c000020	1	0.00 MB	0.00 MB
Select	0x7f4d60000020	1	0.00 MB	0.00 MB
Select	0x7f4d54000020	1	0.00 MB	0.00 MB
Select	0x7f4d4c000020	1	0.00 MB	0.00 MB
Select	0x7f4d50000020	1	0.00 MB	0.00 MB
Select	0x7f4d44000020	1	0.00 MB	0.00 MB
Select	0x7f4d48000020	1	0.00 MB	0.00 MB
Select	0x7f4d3c000020	1	0.00 MB	0.00 MB
Select	0x7f4d40000020	1	0.00 MB	0.00 MB
Select	0x7f4d34000020	1	0.00 MB	0.00 MB
Select	0x7f4d2c000020	1	0.00 MB	0.00 MB
Select	0x7f4d30000020	1	0.00 MB	0.00 MB
Select	0x7f4d24000020	1	0.00 MB	0.00 MB

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



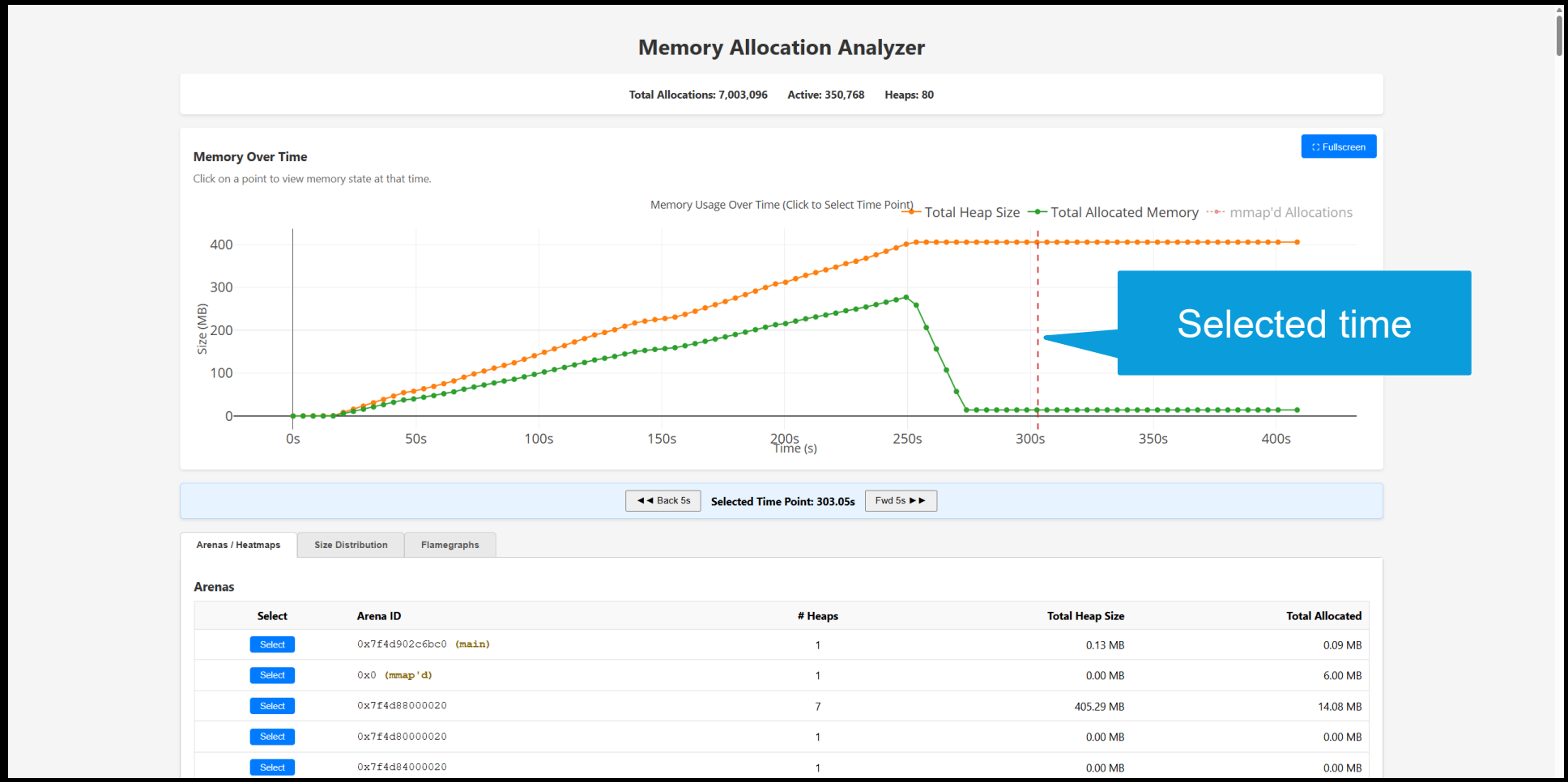
TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



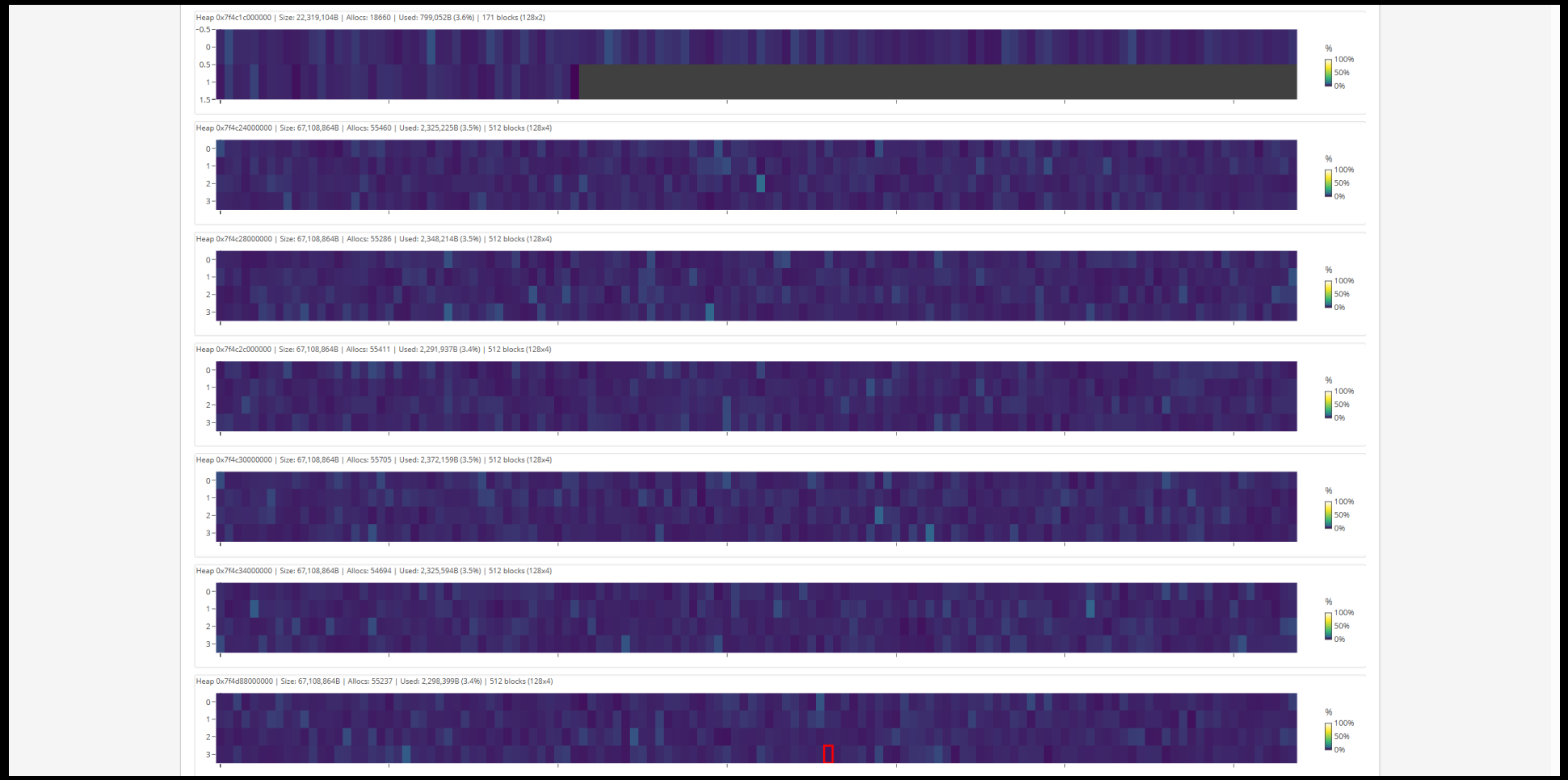
TechAtBloomberg.com

Bloomberg

Engineering

© 2026 Bloomberg Finance L.P. All rights reserved.

Let’s Build a Positional Heap Profiler: Visualizing Pinning



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Let’s Build a Positional Heap Profiler: Visualizing Pinning



Block Allocation Details

Heap 0x7f4d88000000 Block 1067894216 (0x7f4d8b900000 - 0x7f4d8b91ffff): 95 allocations

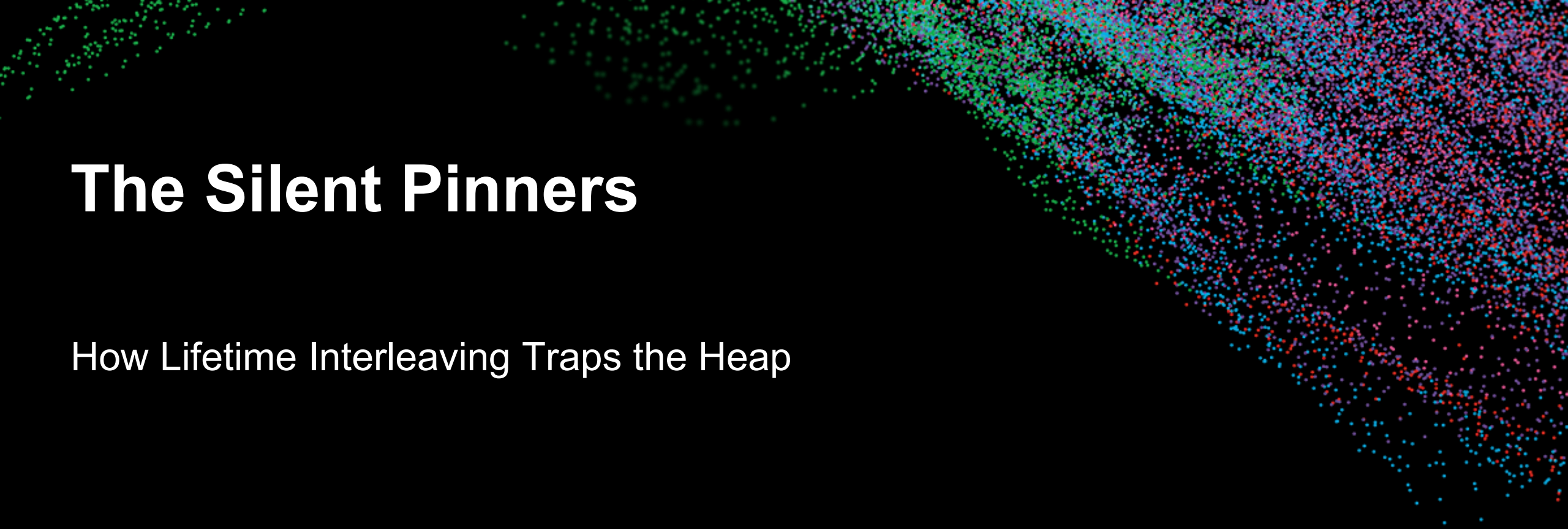
#	Address	Size	Callstack	Free Callstack
1	0x7f4d8b9014a0	24 bytes	__GI___libc_malloc operator new(unsigned long) (anonymous namespace)::collect_tokens_v(unsigned long) (anonymous namespace)::parse_json(unsigned long, unsigned int) (anonymous namespace)::dispatch_alloc(unsigned long, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::serve_request((anonymous namespace)::Worker&, unsigned int, unsigned int, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::worker_loop((anonymous namespace)::Worker*) void std::__invoke_impl<void, void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(std::__invoke_memfun_deref, void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) std::__invoke_result<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>::type std::__invoke<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) void std::thread::_Invoker<std::tuple<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*> >::_M_invoke<0ul, 1ul, 2ul>(std::_Index_tuple<0ul, 1ul, 2ul>)	N/A
2	0x7f4d8b9014c0	100 bytes	__GI___libc_malloc operator new(unsigned long) std::_Vector_base<char, std::allocator<char> >::_M_allocate(unsigned long) std::_Vector_base<char, std::allocator<char> >::_M_create_storage(unsigned long) std::_Vector_base<char, std::allocator<char> >::_Vector_base(unsigned long, std::allocator<char> const&) std::vector<char, std::allocator<char> >::vector(unsigned long, std::allocator<char> const&) (anonymous namespace)::vector_leaf(unsigned long) (anonymous namespace)::collect_tokens_v(unsigned long) (anonymous namespace)::parse_json(unsigned long, unsigned int) (anonymous namespace)::dispatch_alloc(unsigned long, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&)	N/A
3	0x7f4d8b901670	2 bytes	__GI___libc_malloc operator new(unsigned long) (anonymous namespace)::make_widget(unsigned long) (anonymous namespace)::render_page(unsigned long, unsigned int) (anonymous namespace)::dispatch_alloc(unsigned long, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::serve_request((anonymous namespace)::Worker&, unsigned int, unsigned int, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::worker_loop((anonymous namespace)::Worker*) void std::__invoke_impl<void, void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(std::__invoke_memfun_deref, void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) std::__invoke_result<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>::type std::__invoke<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) void std::thread::_Invoker<std::tuple<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*> >::_M_invoke<0ul, 1ul, 2ul>(std::_Index_tuple<0ul, 1ul, 2ul>)	N/A
4	0x7f4d8b9020f0	32 bytes	__GI___libc_malloc operator new(unsigned long) (anonymous namespace)::make_widget(unsigned long) (anonymous namespace)::render_page(unsigned long, unsigned int) (anonymous namespace)::dispatch_alloc(unsigned long, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::serve_request((anonymous namespace)::Worker&, unsigned int, unsigned int, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::worker_loop((anonymous namespace)::Worker*) void std::__invoke_impl<void, void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(std::__invoke_memfun_deref, void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) std::__invoke_result<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>::type std::__invoke<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) void std::thread::_Invoker<std::tuple<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*> >::_M_invoke<0ul, 1ul, 2ul>(std::_Index_tuple<0ul, 1ul, 2ul>)	N/A
5	0x7f4d8b9032d0	24 bytes	__GI___libc_malloc operator new(unsigned long) (anonymous namespace)::collect_tokens_v(unsigned long) (anonymous namespace)::parse_json(unsigned long, unsigned int) (anonymous namespace)::dispatch_alloc(unsigned long, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::serve_request((anonymous namespace)::Worker&, unsigned int, unsigned int, std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul, 7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>&) (anonymous namespace)::Pool::worker_loop((anonymous namespace)::Worker*) void std::__invoke_impl<void, void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(std::__invoke_memfun_deref, void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) std::__invoke_result<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>::type std::__invoke<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*>(void ((anonymous namespace)::Pool::*&&)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*&&, (anonymous namespace)::Worker*&&) void std::thread::_Invoker<std::tuple<void ((anonymous namespace)::Pool::*)((anonymous namespace)::Worker*), (anonymous namespace)::Pool*, (anonymous namespace)::Worker*> >::_M_invoke<0ul, 1ul, 2ul>(std::_Index_tuple<0ul, 1ul, 2ul>)	N/A

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering



The Silent Pinners

How Lifetime Interleaving Traps the Heap

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Pinners: Object Pools

```
auto JobScheduler::enqueue(std::string_view payload, int priority) -> void
{
    std::shared_ptr<Job> job = d_jobPool.acquire();

    job->setPayload(payload);
    job->setPriority(priority);

    d_threadPool.submit([job]() { job->run(); });
}
```

Application lifetime

© 2026 Bloomberg Finance L.P. All rights reserved.

Pinners: Thread-local objects

```
thread_local std::map<std::string, std::string> ThreadContext::tl_vars;

auto ThreadContext::appendProperty(const std::string& key, const std::string& value) -> void
{
    auto it = tl_vars.find(key);
    if (it != tl_vars.end())
    {
        it->second += ";" + value;
    }
    else
    {
        tl_vars.emplace(key, value);
    }
}
```

Thread lifetime

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Pinners: Locals in Long-Running Functions

```
auto Scheduler::eventLoop() -> void
{
    std::list<ScheduledEvent> timers;

    while (d_running) {
        int nearest = nearestDeadline(timers);

        d_bus.waitUntil(nearest);
        d_bus.pendingSubscriptions(timers);

        for (auto it = timers.begin(); it != timers.end();) {
            it->elapsed += nearest;
            if (it->elapsed >= it->intervalMs) {
                d_bus.fire(*it);
                timers.push_back(reschedule(*it));
                it = timers.erase(it);
            }
            else
                ++it;
        }
    }
}
```

Application lifetime

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Long-lived vs. Transient Objects

- Transient objects: Objects created as part of the handling of some unit of work
- Long-lived objects: Objects whose lifetime that typically matches the application
  - Global/static storage objects, thread-local storage objects, locals in long-running functions (main() included)

Heap pinners!

© 2026 Bloomberg Finance L.P. All rights reserved.

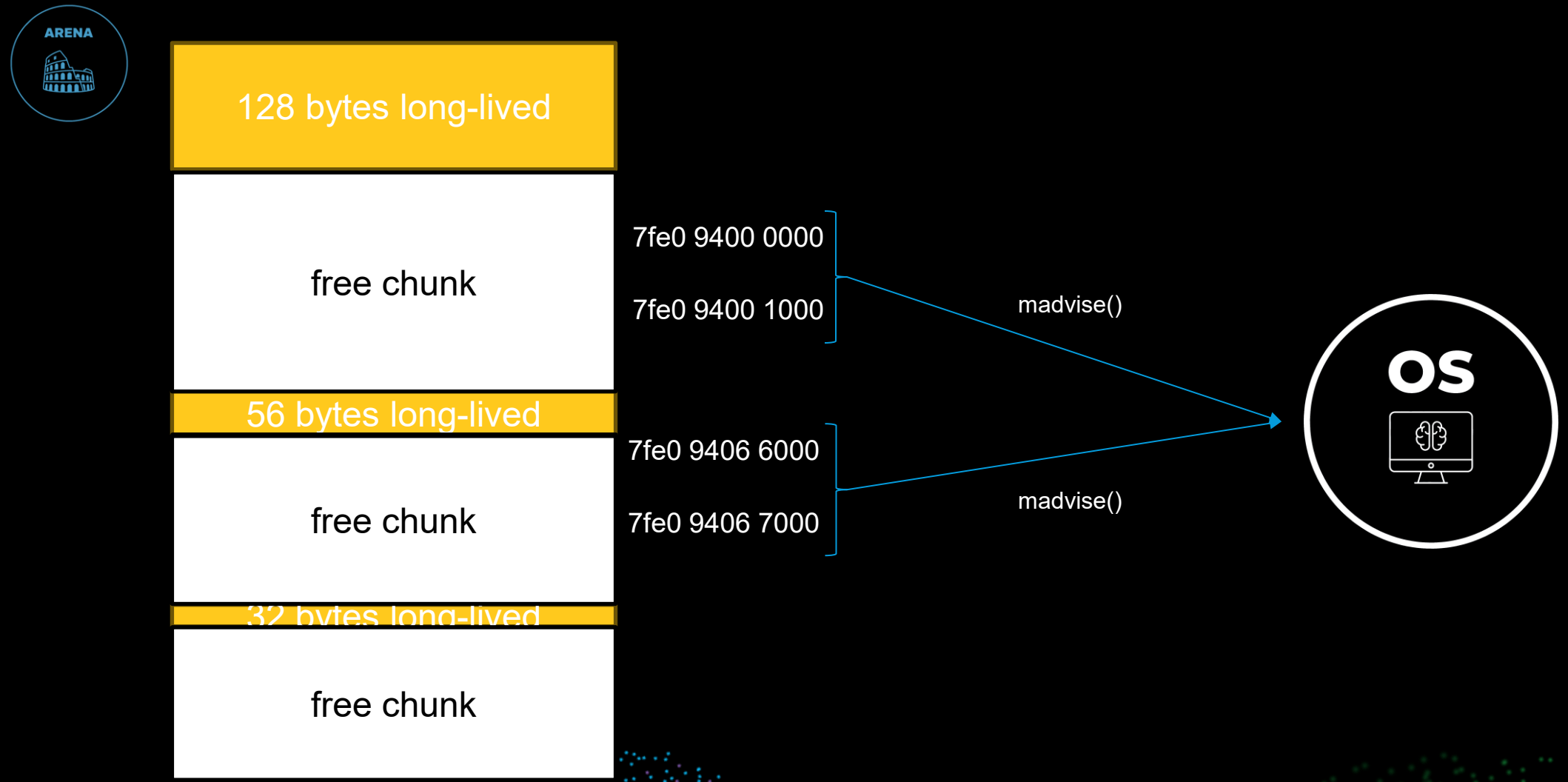
The Quick Hacks

And Why They May Fail

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Treating the Symptom, Not the Cause: *malloc_trim()*



Swapping Allocators: What about *jemalloc*?

- Alternative/drop-in replacement for glibc's *malloc*
- More granular memory segments (4KB-28KB extents vs. 64MB heaps)
- Extents are independent, no heap list, no top chunk
- The underlying problem remains (i.e., takes a single allocation to pin an extent)

© 2026 Bloomberg Finance L.P. All rights reserved.

Architectural Solutions

Lifetime Segregation with *std::pmr* and Allocator-Aware Types

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Separate Transient Objects from Long-Lived Ones

- Transient objects and long-lived objects should be kept on different places
- Transient allocations belong in the general-purpose heap
- Long-lived allocations should go on a different resource

© 2026 Bloomberg Finance L.P. All rights reserved.

The Journey of C++ Allocators

- Pre-C++98: Designed for STL containers to abstract hardware memory model (e.g., 16-bit segmented near/far/huge pointers)
- C++98-C++11: Repurposed by developers for custom allocation strategies (e.g., memory pools, monotonic buffers)
- C++11: Allocators were given state via *std::allocator_traits*
- C++17: Type incompatibility solved with polymorphic allocators *(std::pmr::polymorphic_allocator)*
  - Added *pmr::vector, pmr::deque, pmr::(forward_)list, pmr::(multi)map, pmr::(multi)set, pmr::multiset, pmr::unordered_(multi)map, pmr::unordered_(multi)set, pmr::(w|u16|u32)string*

© 2026 Bloomberg Finance L.P. All rights reserved.

Allocator-Aware Types: Class Members

- Classes should take an allocator in all constructors; the passed allocator must be the same used to construct the object
- Classes should pass this allocator to all their data members; they should too be allocator-ware

```
class UserSession final {
    std::pmr::string username;
    std::pmr::vector<uint64_t> permission

  public:
    explicit UserSession(std::string_view user_name,
                         std::pmr::polymorphic_allocator<> alloc = {}) ame} {}
        : username{user_name, alloc}, permissions{alloc} {}
};
```

Allocator parameter

Passed on to members

```
auto makeUserSession(std::pmr::polymorphi
    std::string_view user_name)
    -> std::shared_ptr<UserSession>
{
    return std::allocate_shared<UserSession>(alloc, user_name, alloc););
}
```

Same allocator to allocate and pass

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Allocator-Aware Types: Conventions

- Classes should tell containers that they are allocator-aware

```cpp
class UserSession final
{
    std::pmr::string username;
    std::pmr::vector<uint64_t> permissions;

  public:
    using allocator_type = std::pmr::polymorphic_allocator<>;

    explicit UserSession(std::string_view user_name,
        std::pmr::polymorphic_allocator<> alloc = {})
        : username{user_name, alloc}, permissions{alloc} {}

    UserSession(const UserSession& other, allocator_type alloc = {});
    UserSession(UserSession&& other, allocator_type alloc);
    auto operator=(const UserSession& other) -> UserSession&
    auto operator=(UserSession&& other) -> UserSession

    auto get_allocator() const noexcept -> allocator_t
        return username.get_allocator();
    }
};
```

typedef/using

Allocator-extended copy/move

For allocator queries

```cpp
rSessionCollection final

    std::pmr::vector<UserSession> sessions;
public:
    using allocator_type = std::pmr::poly

    explicit UserSessionCollection(
        : sessions{alloc} {}

    UserSessionCollection(const UserSessionCollection& other,
        allocator_type alloc = {});
    UserSessionCollection(UserSessionCollection&& other, allocator_type alloc);
    ator=(const UserSessionCollection& other) -> UserSessionCollection&;
    ator=(UserSessionCollection&& other) -> UserSessionCollection&;

    _allocator() const noexcept -> allocator_type {
        return sessions.get_allocator();
    }

};
```

UserSession vector

Allocator now propagated automatically

Allocator-Aware Types: Local Scopes

- Function scope objects have shorter lifetimes, they should use the *default* allocator
- If possible, bump allocators (e.g., *std::monotonic_buffer_resource*) should be used

```cpp
auto EventLoop::processEvent() -> void
{
    std::byte buf[k_SCRATCH_BUFFER_SIZE];
    std::pmr::monotonic_buffer_resource arena{buf, sizeof(buf)};

    auto event = d_queue.pop();
    auto response = std::pmr::string{&arena};
    response += "event type=";
    response += event.payload();

    auto handler = RequestHandler{&arena};
    handler.dispatch(response);

    Logger::instance().log(response);
}
```



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Allocator-Aware Types: “Worker-Loop” Scopes

- Objects with function scope should use the instance allocator
- Objects inside the infinite loop should use the *default* allocator
- “Worker-Loop” free functions should take an allocator parameter
- Functions calling “Worker-Loop” functions inherit the function scope

© 2026 Bloomberg Finance L.P. All rights reserved.

Allocator-Aware Types: Factory Functions

- Factory functions don’t know the lifetime of their created objects, they should take an allocator parameter
- If the created object is allocator-aware, it should be an output parameter
- When building objects to be owned by a different class instance, that

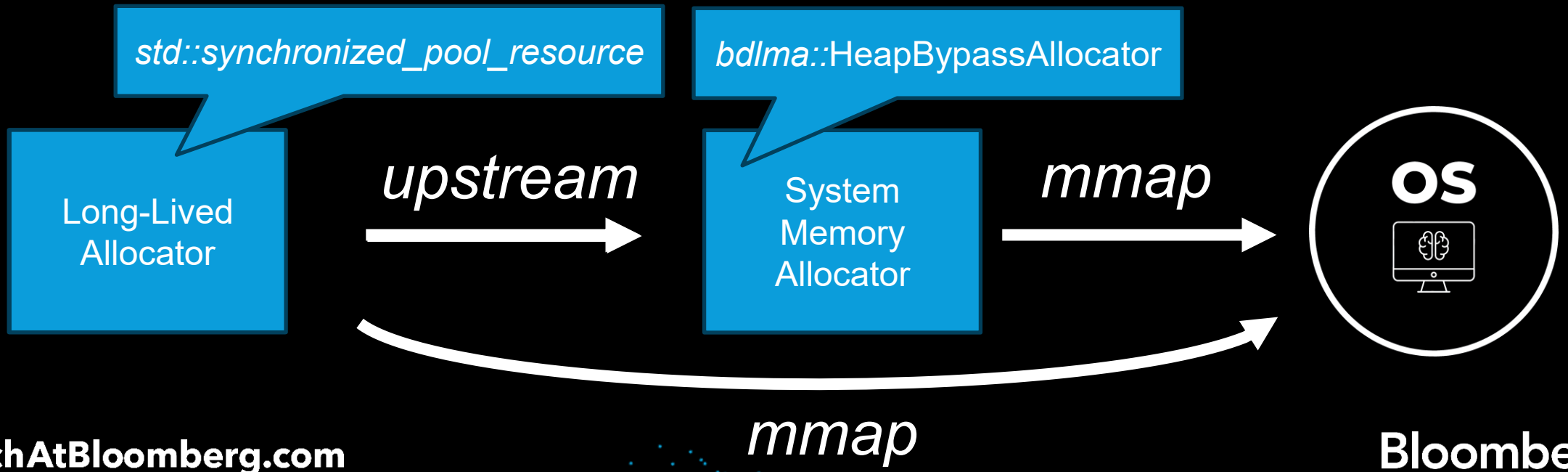
```
auto makeRequestHandler(std::pmr::polymorphic_allocator<> alloc) -> std::shared_ptr<RequestHandler>
{
    return std::allocate_shared<RequestHandler>(alloc);
}
```

Allocator to allocate and pass to constructor

© 2026 Bloomberg Finance L.P. All rights reserved.

Long-Lived Allocator: Implementation

- The *Long-Lived* allocator should not use the general-purpose heap
  - Can chain a system memory allocator to a pooled allocator



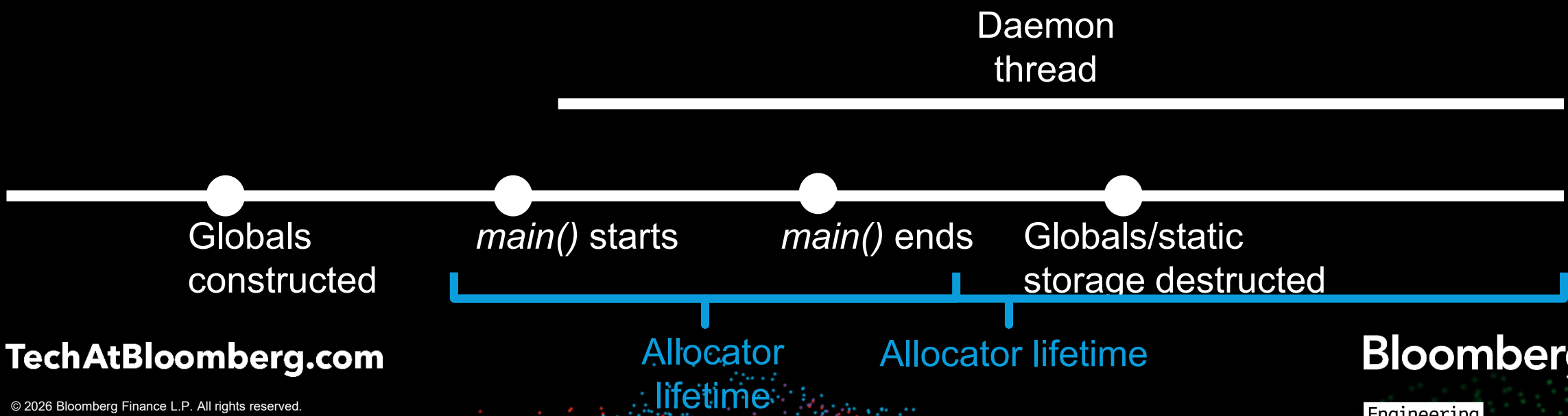
© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Long-Lived Allocator: Visibility & Lifetime

- It should be part of the ambient state
- It should be created and installed at the beginning of *main()*
- It should have a lifetime longer than any potential dependent code



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Long-Lived Allocator: Users

- Singletons, Static/Thread-local storage objects should use the *Long-Lived* allocator
- Code creating long-running threads/coroutines should pass the *Long-Lived* allocator to the function/class constructor
- *main()* scope objects should use the *Long-Lived* allocator

```
auto main() -> int
{
    auto signals = SignalHandler{get_long_lived_resource()};
    auto eventLoop = EventLoop{get_long_lived_resource()};

    Logger::instance().log("EventLoop starting");

    eventLoop.dispatchEvents();

    signals.reset();
    Logger::instance().log("EventLoop shutting down");
}
```

Use Long-Lived Allocator

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Wrapping Up

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Key Takeaways

- Memory footprint ∝ sum of live allocations
- Know thy profiling tools
- Some memory problems require gathering positional information from allocations
- C++ allocators & allocator-aware types are a powerful combination
  - Remember some local objects are “members in disguise”
- Keep objects with different lifetimes separate

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Bloomberg

Engineering

Interested in C++ @ Bloomberg?
We are hiring: bloomberg.com/engineering

Career Opportunities

Learn more about our work

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.